

▼ Вградени структури от данни в Python list и tuple. Цикли.

▼ Листи в Python.

Листите са структура от данни, използвана за съхранение на множество данни в една променлива.

Примерно създаване на лист:

```
1 my_list = ['Hello!', 'Greetings!', 'Hi!']
2 print('my_list: ', my_list)
3 print('Type of my_list: ', type(my_list))
```

```
my_list: ['Hello!', 'Greetings!', 'Hi!']
Type of my_list: <class 'list'>
```

Листите имат дължина (броят на елементите, които съдържат).

```
1 print(len(my_list))
```

```
3
```

В Python, елементите на лист могат да бъдат както от еднакви (в горния пример), така и от различни типове данни:

```
1 my_list1 = ['John Doe', 24, True, 5.9]
2 print(my_list1)
```

```
['John Doe', 24, True, 5.9]
```

▼ Индексиране на лист.

В Python, както и в други програмни езици можем да достъпваме/променяме информацията в съставните части на листа посредством индексиране:

```
1 my_list = ['John', 'Jack', 'Jane']
2 print(my_list[0])
```

```
3 my_list[2] = 'Jean'
4 print('Updated list: ', my_list)
   John
   Updated list: ['John', 'Jack', 'Jean']
```

▼ Slicing на лист.

List в Python също поддържа синтаксиса за slicing. Освен взимане на данните, чиите индекси отговарят на някакво условие за обхват (условие, определено от slicing-а, който прилагаме), тук можем и да променяме тези стойности.

```
1 fruits = ['apple', 'watermelon', 'strawberry', 'pineapple', 'peach', 'cherry']
2
3 print('All fruits:', fruits)
4 print('Selected fruits:', fruits[:5:2])
5
6 fruits[2:4] = ['banana', 'mango']
7 print('Updated fruits:', fruits)

All fruits: ['apple', 'watermelon', 'strawberry', 'pineapple', 'peach', 'cherry']
Selected fruits: ['apple', 'strawberry', 'peach']
Updated fruits: ['apple', 'watermelon', 'banana', 'mango', 'peach', 'cherry']
```

Нещо, което трябва да се отбележи тук е, че в момента замества 2 елемента с 2 други, но не е задължително броят на заместваните елементи да е равен на този на заместващите.

```
1 names = ['Jack', 'John', 'Jill']
2
3 print(f'The list names has values: {names} and has {len(names)} items.')
4
5 names[1:] = ['Bob', 'Mary', 'Julia']
6
7 print(f'After the first update, the list names has values: {names} and \
8 has {len(names)} items.')
9
10 names[1:10] = ['Karen']
11
12 print(f'After the second update, the list names has values: {names} and \
13 has {len(names)} items.')
```

The list names has values: ['Jack', 'John', 'Jill'] and has 3 items.
After the first update, the list names has values: ['Jack', 'Bob', 'Mary', 'Julia'] and
After the second update, the list names has values: ['Jack', 'Karen'] and has 2 items.



▼ Добавяне на елементи към масив

Вероятно най-често използваният метод за добавяне на елементи към масив е методът `append()`.

```
1 my_list = ['cheese', 'bread', 'wine']
2 print(my_list)
3
4 my_list.append('ham')
5 print(my_list)
```

```
['cheese', 'bread', 'wine']
['cheese', 'bread', 'wine', 'ham']
```

Добавяне на елемент на определен индекс.

```
1 positions = ['first', 'third', 'fourth', 'fifth']
2 print(positions)
3
4 positions.insert(1, 'second')
5 print(positions)
```

```
['first', 'third', 'fourth', 'fifth']
['first', 'second', 'third', 'fourth', 'fifth']
```

"Удължаване" на лист.

Методът `extend` може да "долепи"(`append`) не само лист, но и други итерируеми обекти (`tuple`, `set`, `dictionary` и т.н.)

```
1 first_list = [1, 2, 3, 4, 5]
2 second_list = [6, 7, 8]
3
4 print('First list: ', first_list)
5 print('Second list: ', second_list)
6
7 first_list.extend(second_list)
8
9 print('Extended first list: ', first_list)
```

```
First list: [1, 2, 3, 4, 5]
Second list: [6, 7, 8]
Extended first list: [1, 2, 3, 4, 5, 6, 7, 8]
```

▼ Премахване на елементи от лист.

Премахване първото срещане на даден елемент:

```
1 my_list = ['apple', 'orange', 'peach', 'banana', 'orange']
2
3 print(my_list)
4
5 my_list.remove('orange')
6
7 print(my_list)

['apple', 'orange', 'peach', 'banana', 'orange']
['apple', 'peach', 'banana', 'orange']
```

▼ Методът .pop().

pop() премахва елемент на даден индекс (ако не е посочен индекс, който да бъде премахнат по подразбиране премахва последния елемент) и връща стойността му.

```
1 my_list = ['apple', 'orange', 'peach', 'banana']
2 print(my_list)
3
4 popped_element = my_list.pop(2)
5 print(my_list)
6 print(popped_element)

['apple', 'orange', 'peach', 'banana']
['apple', 'orange', 'banana']
peach
```

```
1 my_list = ['apple', 'orange', 'peach', 'banana']
2 print(my_list)
3
4 popped_element = my_list.pop()
5 print(my_list)
6 print(popped_element)

['apple', 'orange', 'peach', 'banana']
['apple', 'orange', 'peach']
banana
```

▼ Ключовата дума del.

Ключовата дума del може да премахне както определен елемент от лист, така и целия лист. Виждаме, че при изтриване на листа, когато се опитаме да го принтираме получаваме грешка.

```
1 my_list = [4.5, True, 3, 'Jack']
2
3 print(my_list)
4
5 del my_list[1]
6
7 print(my_list)
8
9 del my_list
10
11 print(my_list)
```

```
[4.5, True, 3, 'Jack']
[4.5, 3, 'Jack']
```

```
-----
NameError                                Traceback (most recent call last)
<ipython-input-13-7efcb6fb42c3> in <module>
      9 del my_list
     10
--> 11 print(my_list)

NameError: name 'my_list' is not defined
```

SEARCH STACK OVERFLOW

▼ Методът .clear()

Методът .clear() премахва елементите от масив **без** да изтрива променливата.

```
1 my_list = [3, 1, 43, 4]
2 print(my_list)
3
4 my_list.clear()
5 print(my_list)
```

```
[3, 1, 43, 4]
[]
```

Други методи за листи?

В Python листите имат методи:

- `append()` - Добавя елемент в края на листа
- `clear()` - Премахва всички елементи на листа
- `copy()` - Връща копие на листа
- `count()` - Брои колко елемента има листа с дадена стойност
- `extend()` - Добавя елементи в края на листа от друг лист или някакъв итерируем обект
- `index()` - Връща индекса на първия елемент, който има дадена стойност
- `insert()` - Добавя елемент в дадена позиция
- `pop()` - Премахва елемент на дадена позиция и връща стойността му
- `remove()` - Премахва елемент с дадена стойност
- `reverse()` - Нарежда листа в обратен ред
- `sort()` - Сортира масива

▼ Tuples в Python.

Tuples в Python се използват за обединение и съхранение на данни в една променлива. Синтаксиса им се различава от този на листите по това, че се ограждат с кръгли скоби, вместо квадратните на листите.

Основното различно свойство на tuples в Python е, че те са "immutable" - непроменяеми!

```
1 my_tuple = ('Jake', 'Jerry', 'Mike')
2 print(my_tuple)
3 print(type(my_tuple))

('Jake', 'Jerry', 'Mike')
<class 'tuple'>
```

Индексацията на tuples е идентична с тази на листите.

```
1 my_tuple = ('Mike', True, 27, 'male')
2
3 print(my_tuple)
4
5 print(my_tuple[2])

('Mike', True, 27, 'male')
27
```

Тук можем да видим и какво се случва, ако се опитае да променим някой от елементите на tuple.

```
1 my_tuple = ('Mike', True, 27, 'male')
2
3 my_tuple[1] = False
```

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-17-3f0d34f74674> in <module>
      1 my_tuple = ('Mike', True, 27, 'male')
      2
----> 3 my_tuple[1] = False

TypeError: 'tuple' object does not support item assignment
```

SEARCH STACK OVERFLOW

▼ Цикли в Python.

▼ For loop.

For цикълът се използва за итериране по някаква поредица(стринг, лист, tuple, речник или сет). Можем да кажем, че Python for цикълът прилича повече на foreach, от колкото на класическия for цикъл на други програмни езици.

```
1 some_list = [1, 3, 5, 2, 11]
2
3 for number in some_list:
4     print(number**2)

1
9
25
4
121
```

Обхождаме някакъв итерируем обект като създадем помощна променлива, която да приема стойността на текущия негов елемент.

Какво, ако все пак искаме тази променлива да бъде по-конвенционален итератор?

▼ Функция range()

За да изпълним даден код определен брой пъти може да използваме range(). Тази функция ни връща поредица от числа, която по подразбиране започва от 0 със стъпка 1 и приключва в последното число преди подадената стойност.

```
1 for i in range(5):
2     print(i)

0
1
2
3
4
```

Важно е да се отбележи, че range() функцията връща поредица! Ако променяме стойностите на итератора промяната няма да се запази в следващата итерация.

```
1 for number in range(4):
2     print(number, end='    ')
3     number += 2
4     print(number, end='\n\n')

0    2

1    3

2    4

3    5
```

Функцията range() може да приема и други параметри, отнасящи се за началната стойност и стъпката, която правим за преминаване на следваща стойност.

```
1 for num in range(2, 20, 5):
2     print(num)

2
7
12
17
```

▼ Ключови думи continue и break

Както и в други езици, в Python съществуват системните думи `continue` и `break`. С ключовата дума `continue` преминаваме на следващата итерация от цикъла без да изпълняваме кода надолу в тялото на цикъла. `Break` напълно прекратява изпълняването на цикъла.

```
1 my_list = ['Jack', 'John', 'Jill', 'Mike', 'Jake', 'Blake']
2
3 for name in my_list:
4     if name == 'Jill':
5         continue
6     print(name)
```

```
Jack
John
Mike
Jake
Blake
```

```
1 my_list = ['Jack', 'John', 'Jill', 'Mike', 'Jake', 'Blake']
2
3 for name in my_list:
4     if name == 'Jill':
5         break
6     print(name)
```

```
Jack
John
```

▼ While цикъл.

Цикълът `while` изпълнява кода в тялото си докато дадено условие е вярно.

```
1 num = 5
2
3 while(num > 0):
4     print(num, end=' ')
5     num -= 0.5
```

5 4.5 4.0 3.5 3.0 2.5 2.0 1.5 1.0 0.5

Както и при `for` цикъла, така и тук можем да използваме ключовите думи `continue` и `break`.

```
1 num = 0
2
```

```

3 while(num < 10):
4     num += 1
5     if num % 7 == 0:
6         continue
7     print(num, end=' ')
8
1  2  3  4  5  6  8  9  10

```

```

1 num = 0
2
3 while(num < 10):
4     num += 1
5     if num % 7 == 0:
6         break
7     print(num, end=' ')
8
1  2  3  4  5  6

```

Самостоятелни задачи

1. Създайте програма, която извежда първите N числа от редицата на Фибоначи.
2. Създайте програма, която извежда всички елементи на нечетни позиции от даден лист.
3. Създайте програма, която извежда следното при въведено n = 7

```

1
22
333
4444
55555
666666
7777777

```

4. Създайте програма, която, по подаден лист от дробни числа създава tuple, съдържащ квадратите на оригиналния.
5. Създайте игра "морски шах".